

To Begin with the Lab

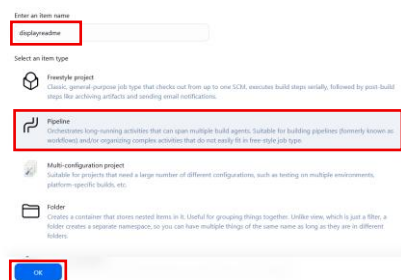
Push and build the docker image to Container Repo

Summary of the Lab

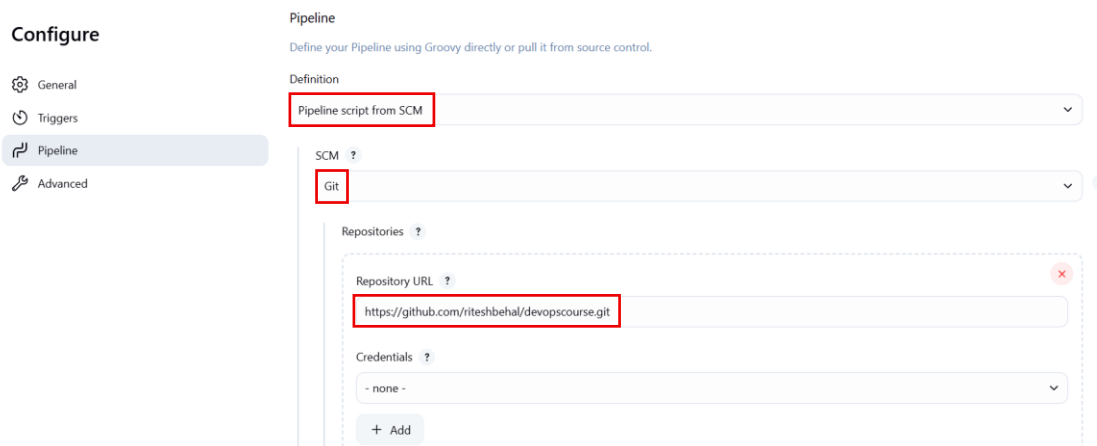
In this lab, a Jenkins pipeline is configured to build and push a Docker image using a Jenkinsfile stored in a subfolder within the devopscourse GitHub repository. A new pipeline job is created, selecting “Pipeline script from SCM” and specifying the Git repository URL, branch (main), and the correct script path (pushdockerimage/Jenkinsfile). After saving, the job is triggered to build the Docker image and push it to the container registry, showcasing Jenkins' integration with Docker and Git for automated CI/CD workflows.

Here we will use the folder pushdockerimage under the repository [devopscourse](#)

- Create a new Item.
- Give the name.
- Select the pipeline option.
- Click OK



- Under the pipeline, in definition, select the option Pipeline script from SCM.
- Under SCM, Select Git.
- Paste the URL of the Github Repository.



- Set the branch to main.
- Give the script path
- Note: give the path properly in the script path.

- In my case the jenkinsfile is under the folder pushdockerimage and inside the devopscourse repository.
- Click Save and click on build now.

Configure

- General
- Triggers
- Pipeline
- Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

***/main**

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add

Script Path ?

pushdockerimage/jenkinsfile

☒ Lightweight checkout ?

Save Apply

Dashboard > pushdockerimage > #1 > Pipeline Overview

#1

Manually run by Aman Rathor Started 2 min 25 sec ago Queued 1 ms Took 15 sec

Graph

```

graph LR
    Start((Start)) --> CheckoutSCM[Checkout SCM]
    CheckoutSCM --> CloneRepo[Clone Repo]
    CloneRepo --> BuildImage[Build Image]
    BuildImage --> DockerLogin[Docker Login]
    DockerLogin --> DockerPush[Docker Push]
    DockerPush --> PostActions[Post Actions]
    PostActions --> End((End))
  
```

Search

- Checkout SCM 0.3s
- Clone Repo 0.52s
- Build Image 7.1s
- Docker Login 0.3s**
- Docker Push 3.8s
- Post Actions 0.32s

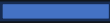
Docker Login 0.3s Started 3m 7s ago Jenkins

```

1 echo $DOCKERHUB_CREDS_PSW | docker login -u $DOCKERHUB_CREDS_USR --password-stdin
2 + docker login -u wolfaman0 --password-stdin
3 WARNING! Your password will be stored unencrypted in /var/lib/jenkins/.docker/config.json.
4 Configure a credential helper to remove this warning. See
5 https://docs.docker.com/engine/reference/commandline/login/#credentials-store
6 Login Succeeded
  
```

```

[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
  
```

Name	Last Pushed 	Contains	Visibility	Scout
 /jenkinstest	5 minutes ago		Public	Inactive
	3 days ago		Public	Inactive
1-2 of 2  				

Trigger One Job from Another

Summary of the Lab

This lab demonstrates how to trigger one Jenkins job from another. First, a pipeline is created that clones a Git repo and reads a README file. Another pipeline job named downstreamjob is also created. Using the Pipeline Syntax Generator, the build step is generated to call downstreamjob. This snippet is added to the original job's script. On running the first job, it automatically triggers the downstream job, illustrating Jenkins' capability to chain jobs and build complex CI/CD flows across multiple tasks.

- Create a new item.
- Give the name of the job
- Select the pipeline
- Click OK.

Dashboard > All > New Item

New Item

Enter an item name

pipelinedemo

Select an item type

-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
-  **Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a

OK

- Under the pipeline, in definition, select the option Pipeline script.
- Select the Hello World

Dashboard > pipelinedemo > Configuration

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

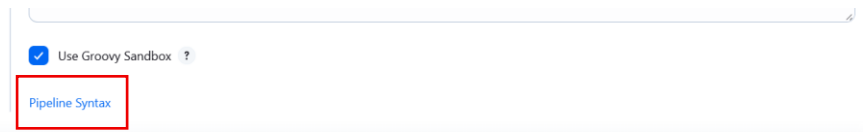
```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Hello') {
6       steps {
7         echo 'Hello World'
8       }
9     }
10  }
11 }
12
```

Use Groovy Sandbox ?

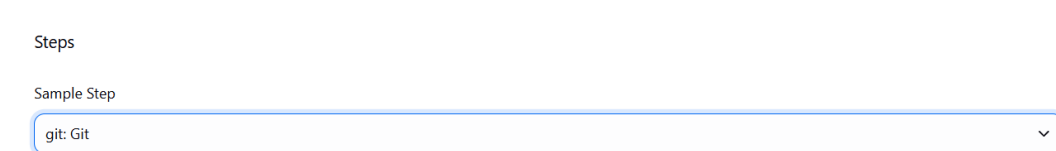
Save Apply

Hello World
try sample Pipeline...
Hello World
GitHub + Maven
Scripted Pipeline

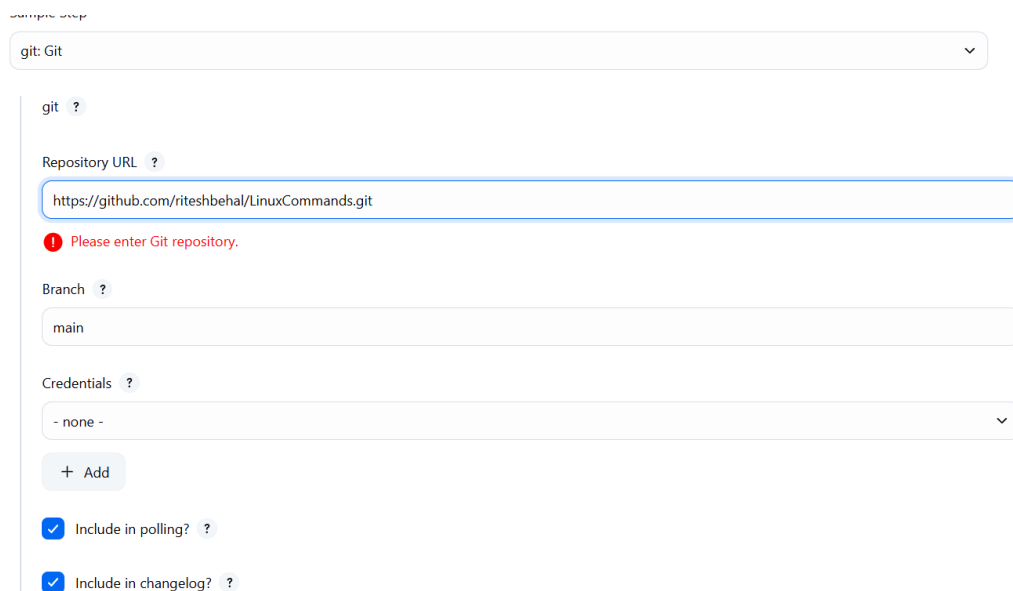
- Now click on the pipeline syntax
- This pipeline syntax is used to generate the code.



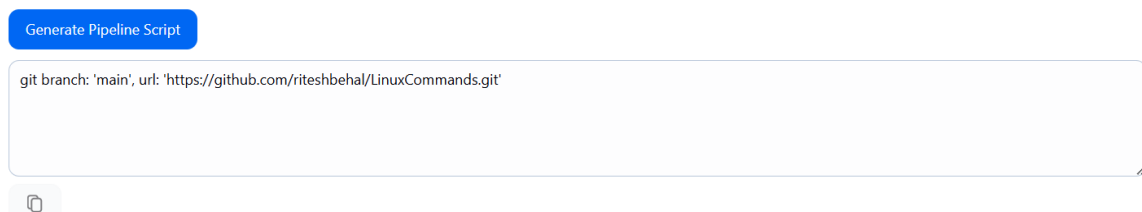
- A new tab will open.
- Now in sampel step, select git: Git



- Now enter the details



- Click generate pipeline script
- It will generate the code.
- Copy the code and paste in the pipeline.



- Now in sampel step, select sh: Shell Script
- Copy this and paste it.

Steps

Sample Step

sh: Shell Script

sh

Shell Script ?

cat README.md

Advanced ▾

Generate Pipeline Script

sh 'cat README.md'

- This is a pipeline script.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Cloning the Git Repo') {
6       steps {
7         git branch: 'main', url: 'https://github.com/riteshbehal/LinuxCommands.git'
8       }
9     }
10    stage('Display README File') {
11      steps {
12        sh 'cat README.md'
13      }
14    }
15  }
```

Hello World

Don't save the job now we have to make changes

Before clicking on the build now

We will create a new job

- Create a new item.
- Give the name of the job
- Select the pipeline
- Click OK.

New Item

Enter an item name

downstreamjob

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a

OK

- Under the pipeline, in definition, select the option Pipeline script.
- Select the Hello World
- Click Save.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Hello') {
6       steps {
7         echo 'Hello World'
8       }
9     }
10  }
11 }
12
```

Hello World

Save

Apply

- Now click on the pipeline syntax
- This pipeline syntax is used to generate the code.

☒ Use Groovy Sandbox ?

Pipeline Syntax

- A new tab will open.
- Now in sample step, select build: Build a job.

Steps

Sample Step

build: Build a job

build ?

- Now enter the job name that is downstreamjob
- Click generate pipeline script.

build ?

Project to Build ?

downstreamjob

☒ Wait for completion ?

☐ Wait until the build starts ?

☒ Propagate errors ?

Quiet period ?

Parameters ?

downstreamjob is not parameterized

Generate Pipeline Script

build 'downstreamjob'



Copy and past the line in the script

Script ?

```
8      }
9      }
10     stage('Display README File') {
11       steps {
12         sh 'cat README.md'
13       }
14     }
15     stage('Trigger Downstream Job') {
16       steps {
17         build 'downstreamjob'
18       }
19     }
20   }
21 }
22 }
```

- Here is the final script.

```
pipeline {
  agent any
```



```

stages {
    stage('Cloning the Git Repo') {
        steps {
            git branch: 'main', url: 'https://github.com/riteshbehal/LinuxCommands.git'
        }
    }

    stage('Display README File') {
        steps {
            sh 'cat README.md'
        }
    }

    stage('Trigger Downstream Job') {
        steps {
            build 'downstreamjob'
        }
    }
}

```

- Now click on Build Now.
- After clicking on Build Now it will create a build in the downstreamjob.

The image displays two screenshots of the Jenkins web interface. The left screenshot shows the 'Pipeline Overview' for a job named 'downstreamjob'. It features a 'Hello' step with a duration of 58ms and a status of 'Completed'. The right screenshot shows the 'Pipeline Overview' for a job named 'pipelinedemo'. It features three steps: 'Cloning the Git Repo' (0.43s), 'Display README File' (0.34s), and 'Trigger Downstream Job' (8.6s). The console log at the bottom right of the right screenshot shows the following output:

```

0 Scheduling project: downstreamjob
1 Starting building: downstreamjob #1
2 Build downstreamjob #1 compl
success

```

Passing Parameter from one Job to another

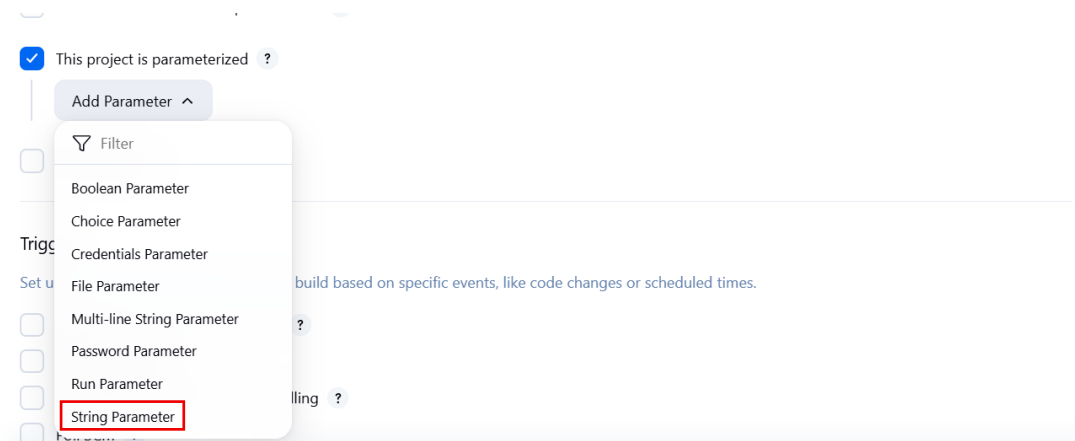
Summary of the Lab

In this lab, the pipeline from the previous lab is enhanced to pass parameters to the downstream job. The downstream job is updated to accept a string parameter (BUILDNUMBER). The Pipeline Syntax Generator is used again to construct a build step with parameters, which is added to the upstream job (pipelinedemo). When triggered, the upstream job passes its build number to the downstream job, which prints it. This demonstrates Jenkins' ability to manage parameterized builds and maintain context across jobs.

- Here we will continue with the previous jobs that we have build 'pipelinedemo and downstreamjob'
- Now go to downstreamjob
- Click configure
- In the general section, tick the option this project is parameterized.



- Now click on Add Parameter
- Select the String Parameter.



String Parameter ?

Name ?

BUILDNUMBER

Default Value ?

Description ?

Plain text [Preview](#)

☐ Trim the string ?

- Now we have to make change in the script.

```
1 pipeline {
2   agent any
3
4   stages {
5     stage('Hello') {
6       steps {
7         echo '$(BUILDNUMBER)'
8       }
9     }
10  }
11 }
12
```

- Now click on the pipeline syntax
- In the sample step select the build: Build a job.
- Enter the job name under the option 'Project to Build'
- In the Parameters;
- Inside the BUILDNUMBER
- Type 'env.BUILD_NUMBER'
- And click generate Pipeline Script.
- Copy the generated syntax.

Parameters ?

BUILDNUMBER

env.BUILD_NUMBER

A list of parameters to pass to the downstream job. When passing secrets to downstream jobs, prefer credentials parameters over password parameters. See [the documentation](#) for details.

(from [Pipeline: Build Step](#))

Generate Pipeline Script

```
build job: 'downstreamjob', parameters: [string(name: 'BUILDNUMBER', value: 'env.BUILD_NUMBER')]
```

📄

- Now we will make change in the script of pipelinedemo job.
- Go to the pipelinedemo
- Click on the configure.
- Now paste the generated syntax in the pipelinedemo script.
- Note: Also in the value, remove the single inverted comma from the env.BUILD_NUMBER and the brackets after build.

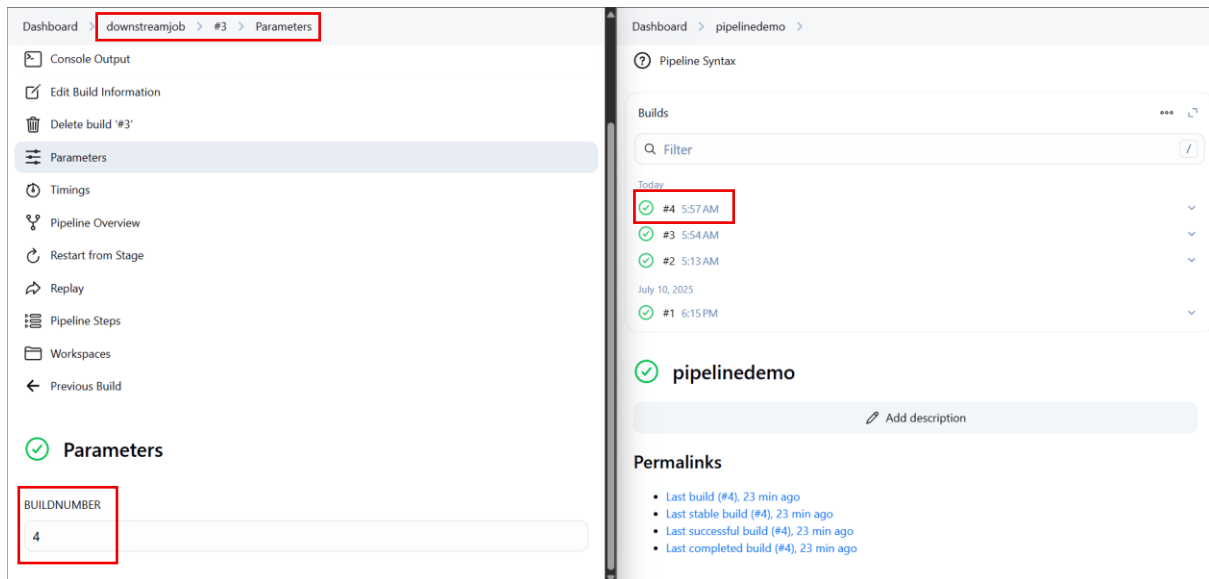
Script ?

```

8      }
9      }
10     stage('Display README File') {
11     steps {
12         sh 'cat README.md'
13     }
14     }
15     stage('Trigger Downstream Job') {
16     steps {
17         build (job: 'downstreamjob', parameters: [string(name: 'BUILDNUMBER', value: env.BUILD_NUMBER)])
18     }
19     }
20 }
21 }
22

```

- Here is output of both the jobs.



- Here is the final script of the pipelinedemo job

```

pipeline {
    agent any

    stages {
        stage('Cloning the Git Repo') {
            steps {
                git branch: 'main', url: 'https://github.com/riteshbehal/LinuxCommands.git'
            }
        }
        stage('Display README File') {
            steps {
                sh 'cat README.md'
            }
        }
        stage('Trigger Downstream Job') {
            steps {
                build (job: 'downstreamjob', parameters: [string(name: 'BUILDNUMBER', value:
env.BUILD_NUMBER)])
            }
        }
    }
}

```

```
    }  
  }  
}
```

-
-
- Here is the final script of the downstream job
-
-

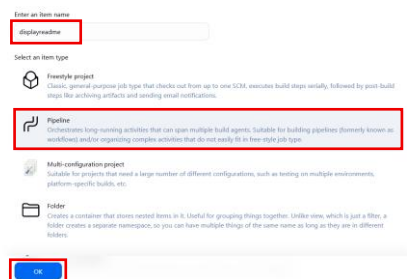
```
pipeline {  
  agent any  
  
  stages {  
    stage('Hello') {  
      steps {  
        echo '${BUILDNUMBER}'  
      }  
    }  
  }  
}
```

Running Shell Script from Jenkins

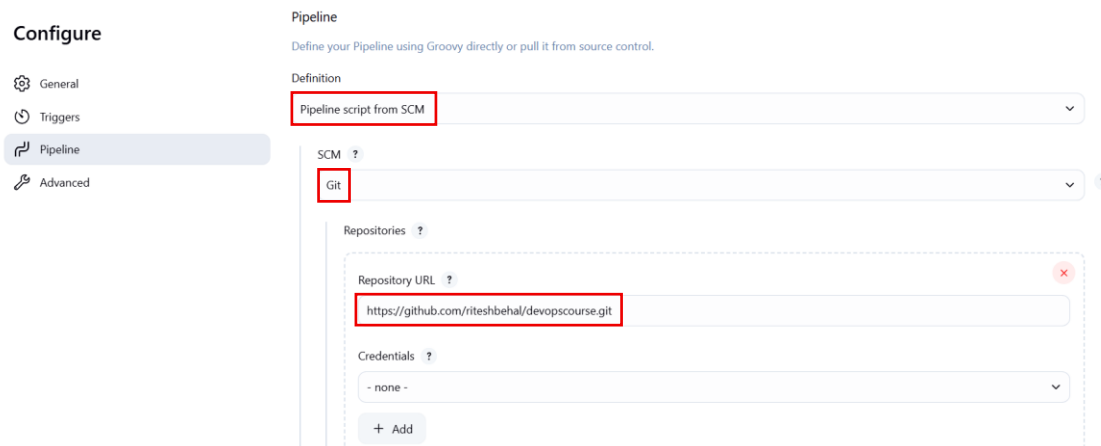
Summary of the Lab

This lab explains how to run a shell script from a Jenkins pipeline using a Jenkinsfile stored in a subdirectory of a GitHub repository. A new pipeline job is created with “Pipeline script from SCM” selected. The Git repo URL, branch name, and correct script path are entered. When the job is triggered, it executes the shell commands specified in the Jenkinsfile. This approach is ideal for running deployment or setup scripts stored in version control, providing seamless integration of scripting with Jenkins CI.

- Create a new Item.
- Give the name.
- Select the pipeline option.
- Click OK



- Under the pipeline, in definition, select the option Pipeline script from SCM.
- Under SCM, Select Git.
- Paste the URL of the Github Repository.



- Set the branch to main.
- Give the script path
- Note: give the path properly in the script path.
- In my case the jenkinsfile is under the folder pushdockerimage and inside the devopscourse repository.
- Click Save and click on build now.

Configure

- General
- Triggers
- Pipeline
- Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

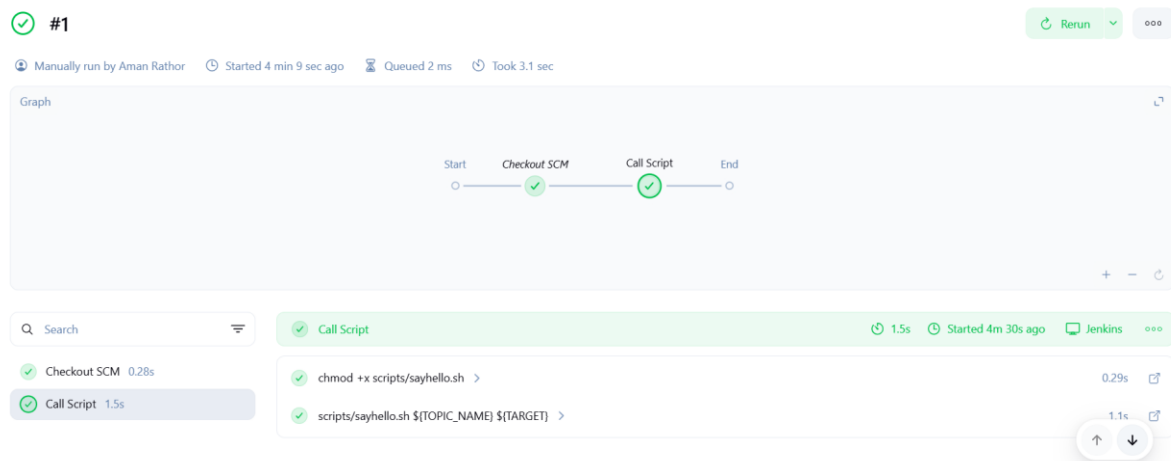
Add

Script Path ?

Jenkinsfile_script

☒ Lightweight checkout ?

- Click on the Build Now.
- Here is the output of the file.



- Here now we will get the different option in place of build now when we create a parameter

Dashboard > runshellsript >

Pipeline runshellsript

This build requires parameters:

TOPIC_NAME

Topics to learn

DevOps

TARGET

Target goal

Dream-Job

Builds

Filter

Today

✓ #1 6:45 AM

Build

Cancel

Click on the build with parameters

And click build.

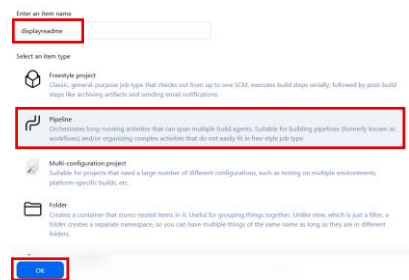
```
+ chmod +x scripts/sayhello.sh
[Pipeline] sh
+ scripts/sayhello.sh DevOps Dream-Job
Learn DevOps , get Dream-Job
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Build and Unit Test using Docker Container Agent

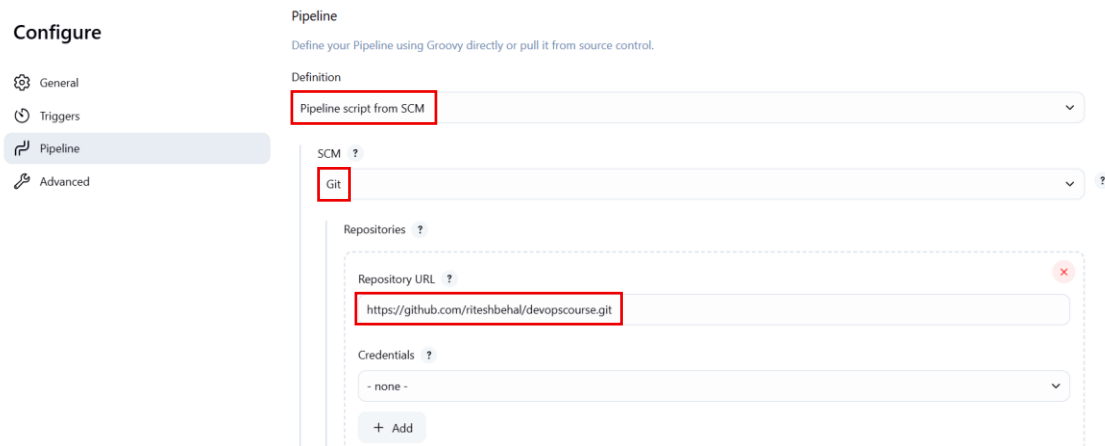
Summary of the Lab

Here, Jenkins is used to run unit tests inside a Docker container using a defined Jenkinsfile. A new pipeline job is created, sourcing the script from GitHub. The Jenkinsfile uses a Docker agent to provide an isolated test environment. Once built, test results can be viewed in Jenkins. To simulate test failure, a deliberate error is introduced in `test_calc.py`, demonstrating Jenkins' ability to detect and report failed unit tests. This reinforces the best practice of testing code in clean, containerized environments.

- Create a new Item.
- Give the name.
- Select the pipeline option.
- Click OK



- Under the pipeline, in definition, select the option Pipeline script from SCM.
- Under SCM, Select Git.
- Paste the URL of the Github Repository.



- Set the branch to main.
- Give the script path
- Note: give the path properly in the script path.
- In my case the jenkinsfile is under the folder pushdockerimage and inside the devopscourse repository.

- Click Save and click on build now.

Configure

- General
- Triggers
- Pipeline
- Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

`*/main`

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

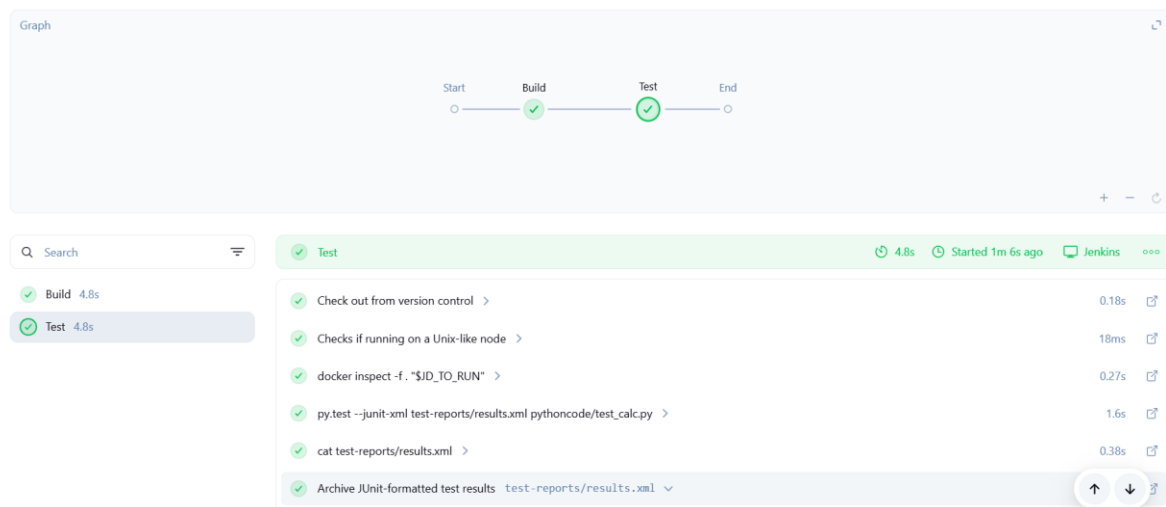
Add

Script Path ?

`pythoncode/Jenkinsfile`

☒ Lightweight checkout ?

- Click on the Build Now.
- Here is the output of the file.



- If you click on Test Results, you can see the results.

- History
- Timings
- Git Build Data
- Test Result
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

Test Result

0 failures

5 tests
Took 24 ms.

Add description

All Tests

Package	Duration	Fail	(diff)	Skip	(diff)	Pass	(diff)	Total	(diff)
pythoncode.test_calc	2 ms	0		0		5	+5	5	+5

- Now let's make one test case fail and check the results again.
- Now I will make the changes in the file test_calc.py on the github.
- I have made the change instead of 3 I have made it 4 because this function is adding two numbers $1+2 = 3$ but I made it 4 now.

```
def test_add_integers(self):
    """
    Test that the addition of two integers returns the correct total
    """
    result = calc.add2(1, 2)
    self.assertEqual(result, 4)
```

- Here is the status part.

Dashboard > runshellsript >

Status

Changes

Build Now

Configure

Delete Pipeline

Stages

Rename

Pipeline Syntax

runshellsript

Latest Test Result (1 failure / +1)

Permalinks

- Last build (#4), 1 min 49 sec ago
- Last stable build (#3), 12 min ago
- Last successful build (#3), 12 min ago
- Last failed build (#4), 1 min 49 sec ago
- Last unsuccessful build (#4), 1 min 49 sec ago
- Last completed build (#4), 1 min 49 sec ago

Add description

Test Result Trend

Passed

Skipped

Failed

- Here are the test results.

History

Timings

Git Build Data

Test Result

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Test Result

1 failures (+1)

5 tests (±0)
Took 33 ms.

Add description

All Failed Tests

Test Name	Duration	Age
+ pythoncode.test_calc.TestCalc.test_add_integers	0 ms	1

All Tests

Package	Duration	Fail	(diff)	Skip	(diff)	Pass	(diff)	Total	(diff)
pythoncode.test_calc	2 ms	1	+1	0		4	-1	5	

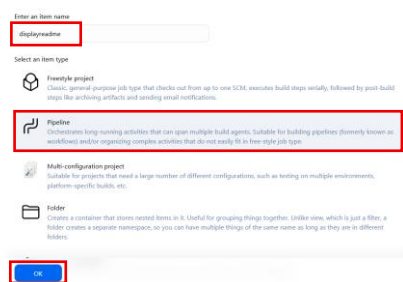
REST API Jenkins 2.504.3

Parallel Stage

Summary of the Lab

This lab shows how Jenkins executes stages in parallel to reduce build time. Two examples are provided: the first uses structured stage blocks under parallel, which display better in the UI; the second uses the parallel() function directly inside a steps block. Both scripts run simulated tests on Windows and Linux by sleeping for 10 or 20 seconds. The lab highlights the difference in execution and display between the two approaches, showing the advantage of using full stage blocks for visibility and maintainability.

- Create a new Item.
- Give the name.
- Select the pipeline option.
- Click OK



- Under the pipeline, in definition, select the option Pipeline script.
- Here is the script for 10 second sleep.

```
pipeline {
    agent any

    stages {
        stage('Run Tests') {
            parallel {
                stage('Test On Windows') {
                    steps {
                        sleep 10
                        echo "run windows tests"
                    }
                }
            }
        }
    }
}
```

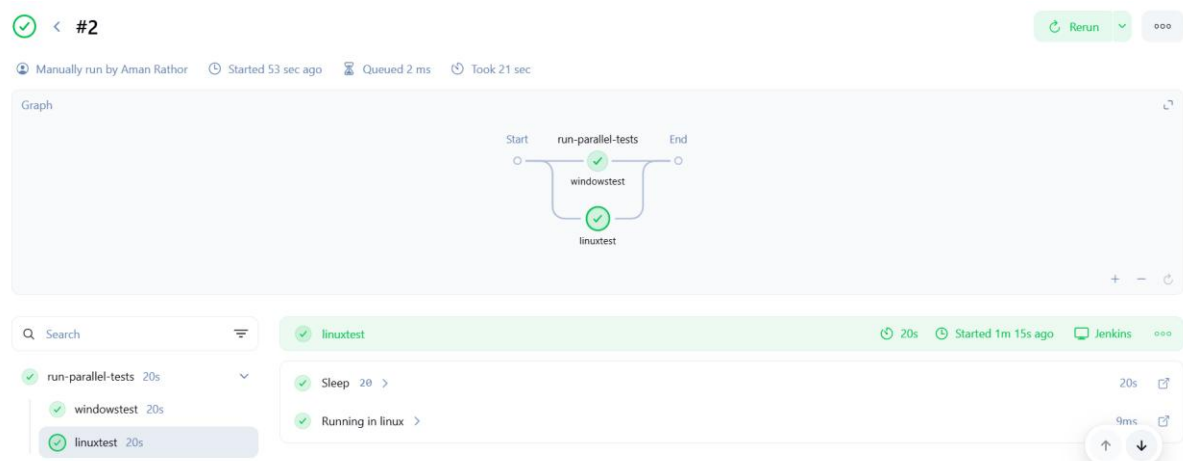
-

```

stages {
  stage('run-parallel-tests') {
    steps {
      parallel(
        windowstest: {
          sleep 20
          echo "Running in windows"
        },
        linuxtest: {
          sleep 20
          echo "Running in linux"
        }
      )
    }
  }
}

```

- This pipeline runs **one stage**: run-parallel-tests.
- Inside that stage, it runs two branches **in parallel**:
 - windowstest: sleeps 20 seconds, then prints "Running in windows"
 - linuxtest: sleeps 20 seconds, then prints "Running in linux"
- Here are the result for the second script



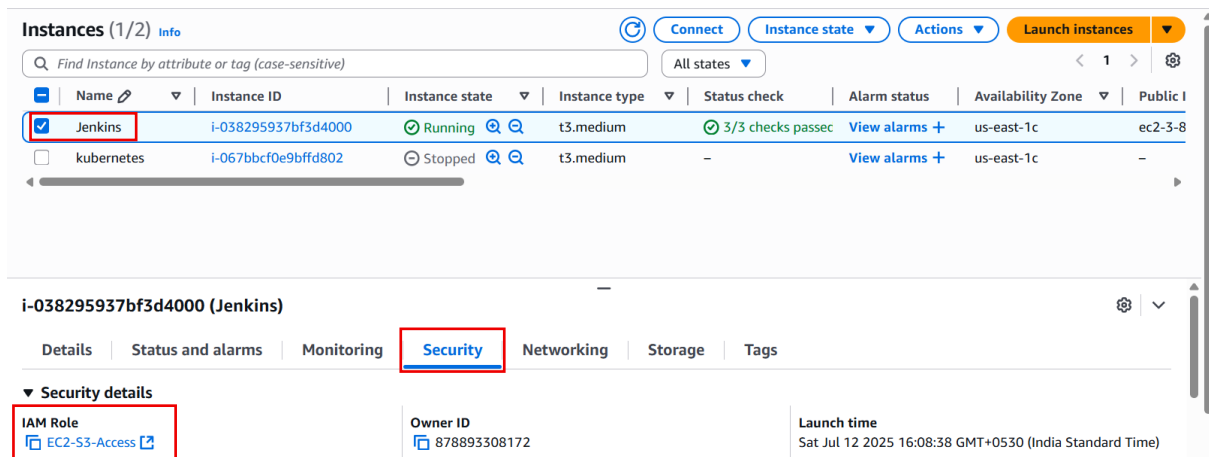
- Here is the difference between both the script.
 - The **first code** uses full stage blocks inside parallel, providing better **UI visibility** and structure.
 - The **second code** uses parallel() directly in steps, which is **less structured** and doesn't show as proper stages in the UI.
 - The first is preferred for **production pipelines**, while the second is better for **quick/test jobs**.

Trigger a lambda from Jenkins

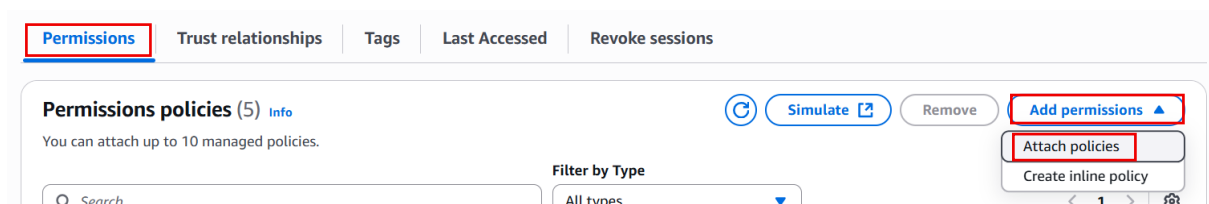
Summary of the Lab

This lab walks through triggering an AWS Lambda function from Jenkins. After assigning the necessary IAM role and permissions to the EC2 instance running Jenkins, a Lambda function is created and deployed using Python. A Jenkins pipeline is configured with two stages: one invokes the Lambda using AWS CLI, and the other displays the output from the response file. This integration allows Jenkins to interact with AWS services seamlessly, enabling automation workflows that include Lambda functions as part of a CI/CD process

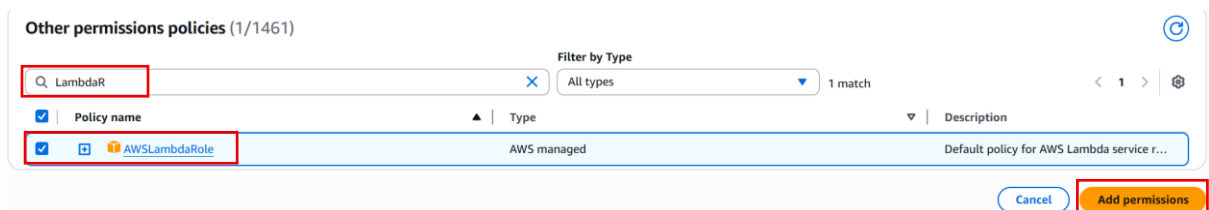
- Here we will create a lambda file and will give the permission for the lambda role.
- Go AWS console.
- Select the instance that you are working on.
- Go to security option.
- Click on the IAM Role.
- Check for IAM Role



- In the permission tab, click on the Add permission
- Click on attach policies.



Search for the policy and click on the Add permission.



- Then we need to create a Lambda function, search and navigate to lambda. Here you just need to choose Author from scratch and give it a name then choose Python 3.11 as your environment and keep rest of the settings to default and create your lambda function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.

☒ x86_64
 ☐ arm64

- Then come to the general configuration and click on edit. Here we are going to increase the timeout limit to 1 minute from just 3 seconds. Click on save to save your changes.

Code | Test | Monitor | **Configuration** | Aliases | Versions

General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables

General configuration [Info](#)

Description -	Memory 128 MB	Ephemeral storage 512 MB
Timeout 0 min 3 sec	SnapStart Info None	

Timeout

min
 sec

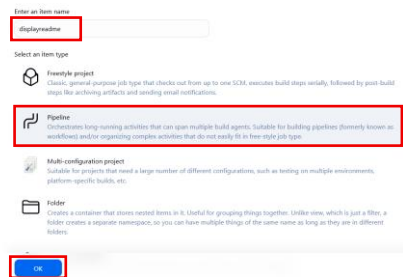
Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☒ Use an existing role
 ☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

[View the moviesPosterDesignPoster-role-sxc0h2hz role](#) on the IAM console.

- Create a new Item.
- Give the name.
- Select the pipeline option.
- Click OK



- Under the pipeline, in definition, select the option Pipeline script.

```
pipeline {
  agent any

  stages {
    stage('Invoke Lambda') {
      steps {
        sh 'aws lambda invoke --function-name hello_lambda response.json --region us-east-1'
      }
    }
    stage('Display Output') {
      steps {
        sh 'cat response.json'
      }
    }
  }
}
```

- Click save
- Click build now.

</> Changes

📄 Console Output

📄 Edit Build Information

🗑 Delete build '#2'

🕒 Timings

🔗 Pipeline Overview

🔄 Restart from Stage

🔄 Replay

☰ Pipeline Steps

📁 Workspaces

⬅ Previous Build

➡ Next Build

Started by user [Aman Rathor](#)

[Pipeline] Start of Pipeline

[Pipeline] node

Running on [Jenkins](#) in /var/lib/jenkins/workspace/lambda

[Pipeline] {

[Pipeline] stage

[Pipeline] { (Invoke Lambda)

[Pipeline] sh

+ aws lambda invoke --function-name hello_lambda response.json --region us-east-1

{

"statusCode": 200,

"executedVersion": "\$LATEST"

}

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (Display Output)

[Pipeline] sh

+ cat response.json

{"statusCode": 200, "body": "\nHello from AWS Lambda\n"}

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

Inserting Manual Approval

Summary of the Lab

- I am using the same lambda job.
- Go to the configure.
- Change the script
- Here is the script

```
pipeline {
  agent any

  stages {
    stage('Manual Approval') {
      steps {
        input message: 'Invoke Lambda? (Click "Proceed" to continue) '
      }
    }
    stage('Invoke Lambda') {
      steps {
        sh 'aws lambda invoke --function-name hello_lambda response.json --region us-east-1'
      }
    }
    stage('Display Output') {
      steps {
        sh 'cat response.json'
      }
    }
  }
}
```

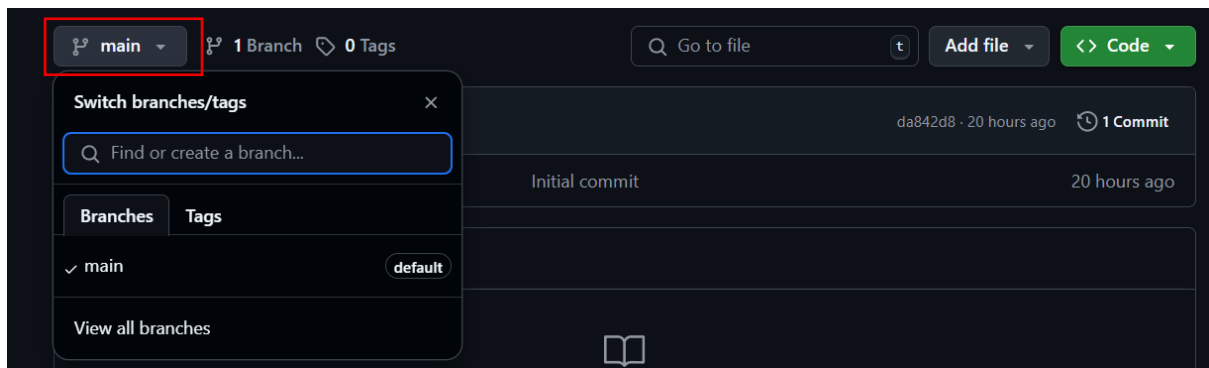
- Here is the output.



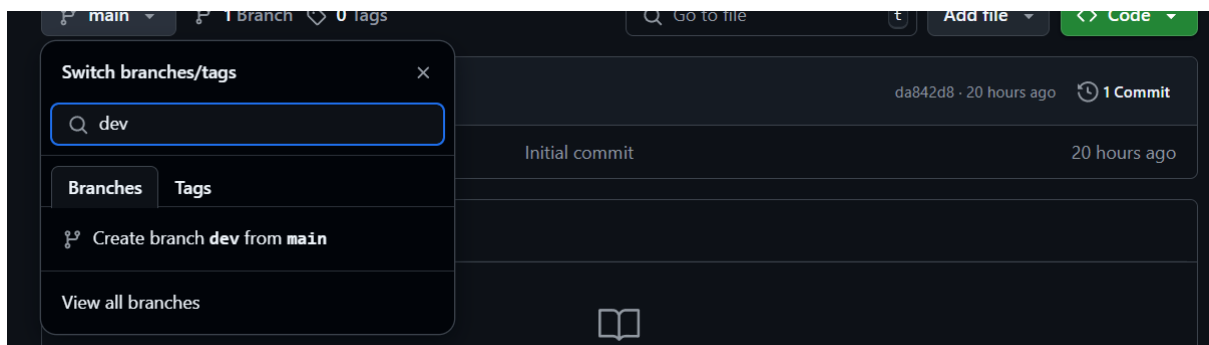
Jenkins Multibranch Pipeline

Summary of the Lab

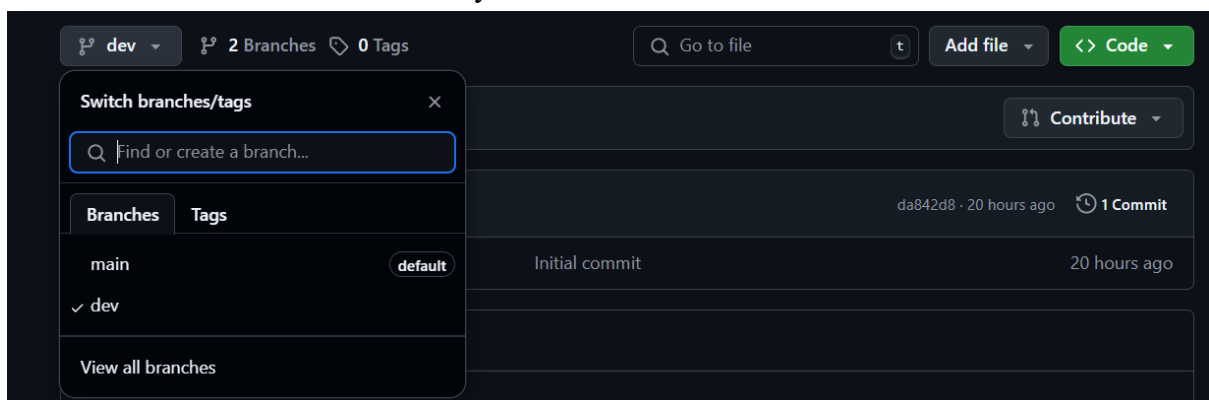
- First you have to create the branches on the GitHub to create the multibranch job.
- Go to GitHub
- Go to main
- Click on it



- Create a branch.
- In the search bar, type the branch name that you want to create.
- Click on create branch from main.



- Like create as many as branch you want.
- dev branch is created successfully.



- Now go the Jenkinsfile if you want to make changes in it.
- Now let's create a multibranch pipeline
- Go to the new item

- Name the job
- Select the option multibranch pipeline
- And click OK.

Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

- In the branch source, paste the URL of your GitHub Repository.

Configuration

- General
- Branch Sources**
- Build Configuration
- Scan Multibranch Pipeline Triggers
- Orphaned Item Strategy
- Appearance
- Health metrics
- Properties

Branch Sources

Git

Project Repository ?
`https://github.com/riteshbehal/multibranchtest.git`

Credentials ?
- none -

+ Add

Behaviors

Discover branches

?

Add

- Choose the path file correctly

Configuration

- General
- Branch Sources**
- Build Configuration
- Scan Multibranch Pipeline Triggers
- Orphaned Item Strategy
- Appearance
- Health metrics
- Properties

Branch Sources

Git

Project Repository ?
`https://github.com/riteshbehal/multibranchtest.git`

Credentials ?
- none -

+ Add

Behaviors

Discover branches

?

Add

- I put 3 mins time for triggering, like if you will add one more branch in your GitHub repository the branch will create automatically after 3 mins.

Status

Configure

Scan Multibranch Pipeline Now

Scan Multibranch Pipeline Log

Multibranch Pipeline Events

Delete Multibranch Pipeline

Check File Fingerprint

Rename

Pipeline Syntax

Credentials

multi-branch

learning jenkins

Branches (4)

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	dev	1 day 19 hr #1	N/A	3.3 sec
✓	☀	main	1 day 19 hr #1	N/A	2.6 sec
✓	☀	Staging	1 day 19 hr #1	N/A	2.5 sec

- Later on, I add the dev branch and I formed after 3 min.

Dashboard > multi-branch >

Status

Configure

Scan Multibranch Pipeline Now

Scan Multibranch Pipeline Log

Multibranch Pipeline Events

Delete Multibranch Pipeline

Build History

Project Relationship

Check File Fingerprint

Rename

Pipeline Syntax

Credentials

multi-branch

learning jenkins

Branches (4)

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	dev	1 day 19 hr #1	N/A	3.3 sec
✓	☀	dev1	1 day 19 hr #1	N/A	1.3 sec
✓	☀	main	1 day 19 hr #1	N/A	2.6 sec
✓	☀	Staging	1 day 19 hr #1	N/A	2.5 sec

Icon: S M L

Build Queue

- You can also build the job separately.

S	W	Name ↓	Last Success	Last Failure	Last Duration
✓	☀	dev	1 day 19 hr #1	N/A	3.3 sec
✓	☀	dev1	19 hr #1	N/A	1.3 sec
✓	☀	main	19 hr #1	N/A	2.6 sec
✓	☀	Staging	19 hr #1	N/A	2.5 sec

</> Changes

▶ Build Now

⚙ View Configuration

📄 Stages

🔍 Pipeline Syntax